Department of Computer Engineering
University of Peradeniya
CO523 - Programming Languages

Lab 09: Functional Programming Paradigm

---

1. Objectives

- Understand the theoretical foundation of functional programming
- Identify and implement pure functions.
- Apply first-class and higher-order functions in solving problems.
- Implement recursion as an alternative to iteration.
- Demonstrate immutability in program design.
- Write small functional programs in Python or Haskell.

2. Functional Programming Paradigm

Functional programming (FP) is a declarative programming paradigm that treats computation as the evaluation of mathematical functions and avoids changing state and mutable data. This approach contrasts with imperative programming, which focuses on how to perform tasks with explicit step-by-step instructions. It is rooted in mathematical theory and has gained popularity for its ability to produce more predictable, maintainable, and bug-free code.

Functional programming has its origins in the lambda calculus, a formal system developed in the 1930s by mathematician Alonzo Church. Lambda calculus is a framework for defining functions and their evaluations and forms the theoretical foundation of functional programming. The first functional programming language, LISP (List Processing), was created in the late 1950s by John McCarthy. LISP introduced many functional programming concepts, such as first-class functions and recursion, which have influenced numerous modern languages.

3. Concepts of Functional Programming

Functional programming is built on several core principles:

3.1. Pure Functions

Pure functions are a core concept in functional programming. A function is considered pure if it always produces the same output for the same input, regardless of when or where it is called. This means that the result of a pure function depends only on its input arguments and not on any external state such as global variables, files, user input, or system time. Because of this property, pure functions are deterministic, which means their behavior is completely predictable.

Another essential characteristic of pure functions is that they have no side effects. They do not modify their input arguments, alter global or local variables, or perform input/output operations such as printing to the screen or writing to a file. This absence of side effects is closely related to the concept of immutability, where data is not changed after it is created. The only outcome of a pure function is the value it returns.

Due to these properties, programs written using pure functions are easier to understand, test, and debug. Since pure functions have no hidden dependencies or side effects, testing them is straightforward: given an input, the output can be directly verified. Additionally, pure functions are highly suitable for parallel and concurrent programming, because they do not share or modify state.

```python
# Python
def pure_function(x, y):
    return x + y

print(pure_function(2, 3))  # Output: 5
print(pure_function(2, 3))  # Output: 5 (same input, same output)
```

Note*: Haskell is a pure functional programming language. You can either install a Haskell compiler locally or use an online compiler (such as https://onecompiler.com/haskell/) to run and experiment with the given examples*

```haskell
-- Haskell

pureFunction :: Int -> Int -> Int
pureFunction x y = x + y

main :: IO ()
main = do
    print (pureFunction 2 3)  -- Output: 5
    print (pureFunction 2 3)  -- Output: 5 (same input, same output)
```

3.2. First-Class and Higher-Order Functions

First-class functions are treated as first-class variable. The first class variables can be passed as arguments to other functions, returned as results, and assigned to variables or stored in data structures.

```python
# Python
def add_one(x):  # add_one is a function
    return x + 1
f = add_one  # add_one is assigned to f
result = f(5)   # f behaves exactly like the function
```

```haskell
-- Haskell

addOne :: Int -> Int
addOne x = x + 1

f :: Int -> Int
f = addOne

-- We define 'result' here
result = f 5

-- main function
main :: IO ()
main = print result
```

Here, addOne is a function and f is just another name for that function. The function used like a variable.

Higher order functions are the functions that take other functions as arguments and they can also return functions.

```python
# Python

def cube(x):        # cube is a function
    return x ** 3

# apply is a higher-order function that takes a function and a value
# it applies the function to the value, and returns the result
def apply(func, x):
    return func(x)

# cube is passed as an argument to apply and applied to 3
result = apply(cube, 3)
print(result)
```

```haskell
-- Haskell

-- Defines cubing logic
cube :: Int -> Int
cube x = x ^ 3

-- Accepts function and value
apply :: (Int -> Int) -> Int -> Int
apply func x = func x
```

```haskell
-- Passes cube into apply
result = apply cube 3

-- Prints the answer
main = print result
```

Map, filter, and reduce are the best examples of higher-order functions because they demonstrate how passing logic as an argument can transform entire data collections.

3.2.1. map

Transforms every item in a collection. It takes a function and applies it to each element individually, returning a new list of the same length.

```python
# Python

nums = [1, 2, 3]
result = list(map(lambda x: x * 2, nums))
print(result) # Output: [2, 4, 6]
```

```haskell
-- Haskell

main = do
    let nums = [1, 2, 3]
    print (map (*2) nums) -- Output: [2, 4, 6]
```

3.2.2. filter

Selects specific items based on a condition. It takes a "predicate" function (one that returns True or False) and keeps only the elements that pass the test.

```python
# Python

nums = [1, 2, 3, 4]
result = list(filter(lambda x: x % 2 == 0, nums))
print(result) # Output: [2, 4]
```

```haskell
-- Haskell

main = print (filter even [1, 2, 3, 4]) -- Output: [2, 4]
```

3.2.3. reduce / fold

Combines all items into a single value. It takes a function that defines how to "merge" two elements and repeats that process until only one result remains.

```python
# Python
```

```python
from functools import reduce
nums = [1, 2, 3, 4]
result = reduce(lambda x, y: x + y, nums)
print(result) # Output: 10
```

```haskell
-- Haskell

main = print (foldl (+) 0 [1, 2, 3, 4]) -- Output: 10
```

3.3. Immutability

Immutability is the concept that data objects cannot be modified after they are created. Instead of changing existing objects, new objects are created with the updated values. This reduces unexpected side effects and makes the code easier to understand and debug.

```python
# Python

# Python tuples are fixed. Once defined, they cannot be changed
coordinates = (40.7128, 74.0060)

# Trying to change an item will cause an ERROR
# coordinates[0] = 34.0522  <-- TypeError: 'tuple' object does not support
item assignment

print(coordinates)
# Output: (40.7128, 74.006)
```

```haskell
-- Haskell

-- Haskell: Data is constant; you create "versions"
main = do
    let numbers = [1, 2, 3]

    -- We create a NEW list using 10 and the "tail" of the old list
    let newNumbers = 10 : tail numbers

    print numbers      -- Output: [1, 2, 3] (The original is still safe!)
    print newNumbers   -- Output: [10, 2, 3]
```

3.4. Recursion

Recursion is a fundamental concept in functional programming, where a function calls itself to solve progressively smaller instances of the same problem. It is widely employed to handle repetitive tasks and serves as a natural alternative to traditional iterative loops, such as for or

while statements, which rely on mutable state. In functional programming, where immutability is emphasized and side effects are minimized, recursion provides a structured mechanism to perform iteration. By repeatedly invoking itself and reducing the problem size with each call, a recursive function continues execution until a well-defined base condition is reached, ensuring termination and correct computation of the desired result.

```python
# Python

def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n - 1)

print(factorial(5))  # Output: 120
```

```haskell
-- Haskell

factorial :: Integer -> Integer
factorial 0 = 1                     -- Base case: 0! is 1
factorial n = n * factorial (n - 1)  -- Recursive case

main = print (factorial 5)           -- Output: 120
```

4. Lab Tasks

4.1. Practice writing pure functions

Write a pure function that:

- Takes a list of numbers
- Returns the average
- Does not modify the original list
- Does not print inside the function

Call the function multiple times with the same input and verify identical output..

4.2. Higher-Order Function Implementation

1. Write a function `apply_twice(func, x)` that applies a function twice.
2. Create functions:

    - square(x) – This function takes single number `x` as input and returns the square of the number.

- increment`(x)` – This function takes a single number `x` as input and returns the value of `x + 1`

3. Pass both functions into `apply_twice`.

Expected behavior:

- `apply_twice(square, 2)` → 16
- `apply_twice(increment, 5)` → 7

4.3. Functional Data Pipeline

nums = [1, 2, 3, 4, 5, 6, 7, 8]

Perform the following in a functional style using above list:

1. Filter even numbers
2. Square them
3. Find the sum

Expected output:
$4^2 + 6^2 + 8^2 = 116$

Students must use:

- `filter`
- `map`
- `reduce` (or `fold` in Haskell)

4.4. Recursion Practice

1. Implement Fibonacci using recursion.

2. Write recursive functions to:

- Reverse a string
- Find the maximum element in a list

Note: *No loops allowed.*

4.5. E-Commerce Discount Engine

You are given a list of product dictionaries (in Python) or a list of tuples (in Haskell). Each product has a name, a category, and a price.

Write a program using the functional programming paradigm (no loops allowed) to:

1. Select only the products belonging to the "Electronics" category.
2. Apply a 10% discount to the prices of those filtered products.
3. Calculate the total cost of all discounted electronics.
4. Write a recursive function to find the product with the highest price in the original list.

Constraints:

- Use only map, filter, and reduce (or fold) appropriately for the data pipeline .
- Data must remain immutable; do not modify the original list .
- The "Highest Price" function must use recursion, not the built-in max() function.
- You may implement your solution in either **Python** or **Haskell**.

```python
# Input Data for Python
products = [
    {"name": "Laptop", "category": "Electronics", "price": 1000},
    {"name": "Shirt", "category": "Clothing", "price": 50},
    {"name": "Phone", "category": "Electronics", "price": 500},
    {"name": "Book", "category": "Media", "price": 20},
    {"name": "Monitor", "category": "Electronics", "price": 300}
]
```

```haskell
-- Input Data for Haskell
-- Define Product as (Name, Category, Price)
products = [ ("Laptop", "Electronics", 1000.0),
             ("Shirt", "Clothing", 50.0),
             ("Phone", "Electronics", 500.0),
             ("Book", "Media", 20.0),
             ("Monitor", "Electronics", 300.0) ]
```

5. Submission

Create a ZIP file containing all your codes and rename it as **CO523_Lab09_EXXYYY.zip**, where EXXYYY represents your E-number.

Upload the Zip file to the FEeLS by the given deadline.